Comprehensive Guide to Convolutional Neural Networks

Core Concepts, Visual Diagrams & Backpropagation Notes

March 29, 2026

Contents

I Essential Guide to Convolutional Neural Networks **5**

1 Introduction to CNNs **5**
1.1 Why CNNs? 5
1.2 CNN Solution 5

2 Convolution Operation **5**
2.1 2D Convolution Mechanics 5
2.1.1 Example: 6×6 Input with 3×3 Kernel 5
2.2 Output Size Formula 6

3 Padding **6**
3.1 Why Padding? 6
3.2 Zero Padding Visualization 7
3.3 Padding Types 7

4 Stride **7**
4.1 Stride Definition 7
4.2 Stride Effects 7

5 3D Convolutions (Multi-Channel) **8**
5.1 RGB Images 8
5.2 3D Convolution Process 8
5.3 Multiple Kernels for Multiple Channels 8

6 Pooling Layers **8**
6.1 Purpose of Pooling 8
6.2 Max Pooling 9
6.3 Average Pooling 9
6.4 Global Average Pooling (GAP) 9

7 Backpropagation in CNNs **9**
7.1 Overview . . . 9
7.2 Gradient for Convolutional Layer . . . 9
7.2.1 Forward Pass . . . 10
7.2.2 Backward Pass: Gradient w.r.t Kernel . . . 10
7.2.3 Backward Pass: Gradient w.r.t Input . . . 10
7.2.4 Backward Pass: Gradient w.r.t Bias . . . 10
7.3 Gradient for Max Pooling . . . 10
7.3.1 Forward Pass . . . 10
7.3.2 Backward Pass . . . 10
7.4 ReLU Activation . . . 11
7.4.1 Forward . . . 11
7.4.2 Backward . . . 11

8 Complete CNN Architecture **11**
8.1 Typical CNN Structure . . . 11
8.2 Example: Simple CNN for MNIST . . . 11
8.3 Key Patterns . . . 12

9 Summary **12**
9.1 Key Formulas . . . 12
9.2 Design Principles . . . 12

10 Detailed Padding Examples **13**
10.1 Padding Calculation for Common Cases . . . 13
10.2 Padding with Stride ¿ 1 . . . 13

11 Advanced Convolution Visualization **14**
11.1 Receptive Field Growth . . . 14
11.2 Parameter Comparison . . . 14

12 Complete Backpropagation Example **14**
12.1 Simple 2-Layer Network . . . 14
12.2 Forward Pass Equations . . . 15
12.3 Backward Pass Equations . . . 15

13 Numerical Example: Small CNN **16**
13.1 Setup . . . 16
13.2 Forward Pass . . . 16
13.3 After ReLU . . . 16
13.4 Backward Pass . . . 17

14 Practical Implementation Tips **17**
14.1 Common Hyperparameters . . . 17
14.2 Architecture Design Rules . . . 17
14.3 Debugging Checklist . . . 17

15 Quick Reference Tables **18**
15.1 Output Size Quick Reference . . . 18
15.2 Receptive Field Quick Reference . . . 18

16 Classic CNN Architectures **18**
16.1 LeNet-5 (1998) . . . 18
16.1.1 LeNet-5 Architecture . . . 19
16.1.2 Layer-by-Layer Breakdown . . . 19
16.1.3 Key Innovations . . . 19
16.1.4 Historical Impact . . . 20
16.2 AlexNet (2012) . . . 20
16.2.1 AlexNet Architecture . . . 21
16.2.2 Detailed Layer Specifications . . . 22
16.2.3 Key Innovations . . . 22
16.3 VGGNet (2014) . . . 22
16.3.1 VGG-16 Architecture . . . 23
16.3.2 Complete VGG-16 Specifications . . . 24
16.3.3 VGG Design Philosophy . . . 24
16.3.4 VGG Variants . . . 25
16.3.5 Key Characteristics . . . 25
16.3.6 Limitations of VGG . . . 25
16.3.7 Legacy and Impact . . . 25

17 Detailed Convolution Mathematics **26**
17.1 Cross-Correlation vs Convolution . . . 26
17.2 Detailed Gradient Derivation . . . 26
17.2.1 Setup . . . 26
17.2.2 Chain Rule Application . . . 26
17.2.3 Gradient w.r.t Input . . . 26
17.3 Numerical Gradient Checking Example . . . 27
17.4 Computational Complexity Analysis . . . 27
17.4.1 Forward Pass Complexity . . . 27
17.4.2 Memory Requirements . . . 28

18 Advanced Pooling Techniques **28**
18.1 Stochastic Pooling . . . 28
18.2 Mixed Pooling . . . 28
18.3 LP Pooling . . . 29
18.4 Spatial Pyramid Pooling (SPP) . . . 29
18.5 Adaptive Pooling . . . 29
18.6 Global Context Pooling (GCP) . . . 29
18.7 Comparison Summary . . . 30

II CNN Backpropagation Notes (With Matrix Layouts) **30**

19 Why a Plain MLP Fails on Raw Image Data **30**

20 How CNN Solves These Issues **31**

21 From Hubel & Wiesel to Modern CNNs (Short History) **31**

22 Layer-0 Input, Kernel, and Layer-1 Output **31**

23 Forward Matrices (the 4 receptive fields) **32**

24 Loss Gradient at Layer-1 **33**

25 Matrix Form for $\partial L/\partial \boldsymbol{W}$ **33**

26 Weight Update Matrix **33**

27 Expanded Scalar Gradients (from chain rule) **34**

28 Padding, Stride, and Output Size Formula **34**

29 Backpropagation Through This Convolution Layer **35**

30 Complete CNN Architecture (Part 2 Reference) **35**

31 Backpropagation in CNNs (General Multi-Channel Form) **36**

32 Pooling Backprop and ReLU Backprop **36**

33 Detailed Padding Examples (Part 2) **36**

34 Receptive Field Growth **37**

35 Classic CNN Architectures Summary (Part 2) **37**

36 Numerical Mini Example and Dying ReLU **37**

37 Advanced Pooling Variants **38**

38 Practical Tips and Quick Reference **38**

Part I
Essential Guide to Convolutional Neural Networks

1 Introduction to CNNs

1.1 Why CNNs?

Traditional fully connected networks fail for images due to:

- **Too many parameters**: A $224 \times 224 \times 3$ image needs $150,528$ input neurons. With 4096 hidden neurons: $150,528 \times 4096 \approx 616$ million weights!
- **No spatial structure**: Flattening destroys the 2D relationship between pixels
- **Not translation invariant**: Same object at different positions treated differently

1.2 CNN Solution

CNNs use two key principles:

1. **Local Connectivity**: Neurons connect only to small local regions
2. **Parameter Sharing**: Same weights (kernel) used across entire image

2 Convolution Operation

2.1 2D Convolution Mechanics

A **kernel** (or filter) is a small matrix of learnable weights that slides over the input.

Operation Steps:

1. Place kernel on input patch
2. Element-wise multiplication
3. Sum all products
4. Move to next position (controlled by stride)

2.1.1 Example: 6×6 Input with 3×3 Kernel

$$\text{Input } I = \begin{bmatrix} 1 & 2 & 3 & 0 & 1 & 2 \\ 4 & 5 & 6 & 1 & 2 & 3 \\ 7 & 8 & 9 & 2 & 3 & 4 \\ 1 & 2 & 3 & 3 & 4 & 5 \\ 4 & 5 & 6 & 4 & 5 & 6 \\ 7 & 8 & 9 & 5 & 6 & 7 \end{bmatrix} \quad \text{Kernel } K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

First position (top-left):

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \odot K = (1 \cdot 1 + 2 \cdot 0 + 3 \cdot -1) + (4 \cdot 1 + 5 \cdot 0 + 6 \cdot -1) + (7 \cdot 1 + 8 \cdot 0 + 9 \cdot -1)$$

$$= (1 + 0 - 3) + (4 + 0 - 6) + (7 + 0 - 9) = -2 + (-2) + (-2) = -6$$

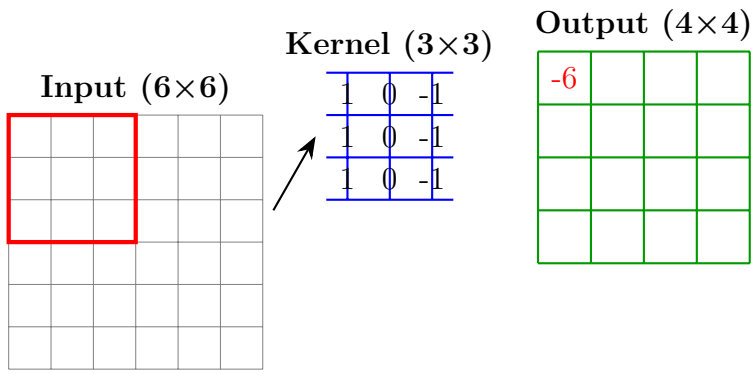


Figure 1: Convolution operation: kernel slides over input to produce output

2.2 Output Size Formula

For input width W, kernel size K, padding P, stride S:

$$\boxed{W_{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1}$$

Example: $W = 6, K = 3, P = 0, S = 1$

$$W_{out} = \left\lfloor \frac{6 - 3 + 0}{1} \right\rfloor + 1 = 3 + 1 = 4$$

3 Padding

3.1 Why Padding?

- **Preserve spatial dimensions**: Without padding, output shrinks
- **Use border information**: Edge pixels are underutilized

3.2 Zero Padding Visualization

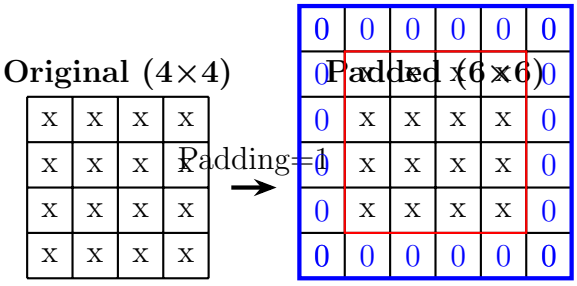


Figure 2: Zero padding adds border of zeros around input

3.3 Padding Types

- **VALID** (No padding): $P = 0$, output shrinks
- **SAME** (Zero padding): $P = (K - 1)/2$, output same size as input (when $S = 1$)

Example with padding:

$$W = 6, K = 3, P = 1, S = 1 \implies W_{out} = \left\lfloor \frac{6 - 3 + 2}{1} \right\rfloor + 1 = 6$$

4 Stride

4.1 Stride Definition

Stride (S) is the number of pixels the kernel moves at each step.

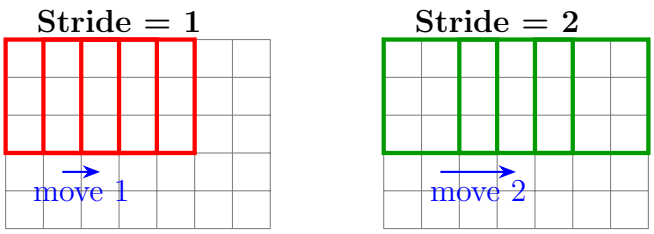


Figure 3: Stride controls how far kernel moves at each step

4.2 Stride Effects

- $S = 1$: Dense sampling, larger output
- $S = 2$: Downsamples by factor of 2, smaller output
- $S > 1$: Alternative to pooling for dimensionality reduction

Example:

$$W = 6, K = 3, P = 0, S = 2 \implies W_{out} = \left\lfloor \frac{6 - 3 + 0}{2} \right\rfloor + 1 = 1 + 1 = 2$$

5 3D Convolutions (Multi-Channel)

5.1 RGB Images

Real images have depth: $H \times W \times C$ where $C = 3$ for RGB.

Key Rule: Kernel depth must match input depth.

$$\text{Input: } H \times W \times C_{in} \quad \text{Kernel: } K \times K \times C_{in}$$

5.2 3D Convolution Process

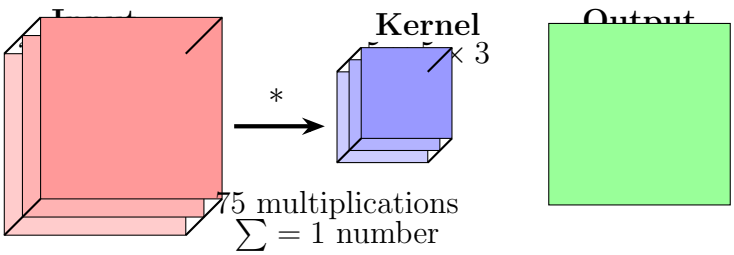


Figure 4: 3D kernel produces 2D output (single channel)

5.3 Multiple Kernels for Multiple Channels

To get output with C_{out} channels: use C_{out} different kernels.

Example: 64 output channels

- Input: $32 \times 32 \times 3$
- 64 kernels, each $5 \times 5 \times 3$
- Each kernel $\rightarrow$ one 28×28 map
- Stack 64 maps $\rightarrow$ Output: $28 \times 28 \times 64$

Total Parameters:

$$\text{Params} = (\text{Kernel Size}) \times (\text{Num Kernels}) = (5 \times 5 \times 3) \times 64 = 4,800$$

6 Pooling Layers

6.1 Purpose of Pooling

- **Reduce spatial dimensions**: Downsample feature maps
- **Translation invariance**: Small shifts don't change output
- **Reduce computation**: Fewer parameters in deeper layers

6.2 Max Pooling

Takes maximum value from each window (most common type).

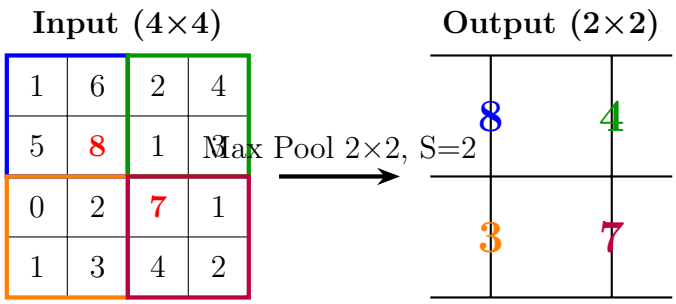


Figure 5: Max pooling: take maximum from each 2×2 region

Common Settings:

- 2×2 window, stride 2: reduces dimensions by half
- 3×3 window, stride 2: overlapping pooling

6.3 Average Pooling

Takes mean of values in each window (less common).

6.4 Global Average Pooling (GAP)

Pools entire spatial dimension to single value per channel.

$$\text{Input: } 7 \times 7 \times 512 \quad \xrightarrow{\text{GAP}} \quad \text{Output: } 1 \times 1 \times 512$$

Used at end of networks (ResNet, etc.) to replace fully connected layers.

7 Backpropagation in CNNs

7.1 Overview

CNNs are trained using backpropagation and gradient descent. Each layer's operation is differentiable, allowing gradients to flow backward.

7.2 Gradient for Convolutional Layer

Given:

- Input: X of shape (H, W, C_{in})
- Kernel: K of shape $(k_H, k_W, C_{in}, C_{out})$
- Output: Y of shape (H', W', C_{out})
- Loss: L

7.2.1 Forward Pass

$$Y_{i,j,c} = \sum_{m=0}^{k_H-1} \sum_{n=0}^{k_W-1} \sum_{d=0}^{C_{in}-1} X_{i+m,j+n,d} \cdot K_{m,n,d,c} + b_c$$

7.2.2 Backward Pass: Gradient w.r.t Kernel

$$\boxed{\frac{\partial L}{\partial K_{m,n,d,c}} = \sum_{i=0}^{H'-1} \sum_{j=0}^{W'-1} X_{i+m,j+n,d} \cdot \frac{\partial L}{\partial Y_{i,j,c}}}$$

Interpretation: Gradient for kernel weight is the convolution of input with upstream gradient.

7.2.3 Backward Pass: Gradient w.r.t Input

$$\boxed{\frac{\partial L}{\partial X_{i,j,d}} = \sum_{m=0}^{k_H-1} \sum_{n=0}^{k_W-1} \sum_{c=0}^{C_{out}-1} K_{m,n,d,c} \cdot \frac{\partial L}{\partial Y_{i-m,j-n,c}}}$$

Interpretation: Gradient for input is the "full" convolution of flipped kernel with upstream gradient.

7.2.4 Backward Pass: Gradient w.r.t Bias

$$\boxed{\frac{\partial L}{\partial b_c} = \sum_{i=0}^{H'-1} \sum_{j=0}^{W'-1} \frac{\partial L}{\partial Y_{i,j,c}}}$$

Interpretation: Bias gradient is sum of all gradients in that channel.

7.3 Gradient for Max Pooling

Max pooling is not differentiable everywhere, but we use sub-gradients.

7.3.1 Forward Pass

$$Y_{i,j,c} = \max_{m,n \in \text{window}} X_{i \cdot S+m, j \cdot S+n, c}$$

During forward pass, we **remember the index** (m^*, n^*) of the maximum value.

7.3.2 Backward Pass

$$\boxed{\frac{\partial L}{\partial X_{i,j,d}} = \begin{cases} \frac{\partial L}{\partial Y_{i',j',d}} & \text{if } X_{i,j,d} \text{ was the max in its pooling window} \\ 0 & \text{otherwise} \end{cases}}$$

Interpretation: Gradient only flows through the "winner" neuron that produced the max. All other neurons get zero gradient.

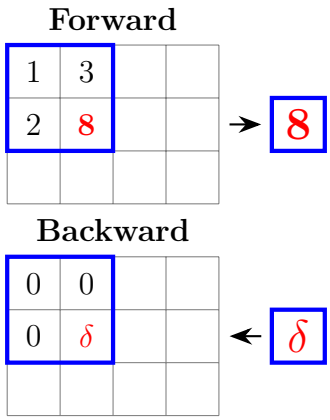


Figure 6: Max pooling backprop: gradient only flows to max position

7.4 ReLU Activation

7.4.1 Forward

$$y = \max(0, x)$$

7.4.2 Backward

$$\boxed{\frac{\partial L}{\partial x} = \begin{cases} \frac{\partial L}{\partial y} & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}}$$

Gradient passes through positive neurons, stops at negative ones.

8 Complete CNN Architecture

8.1 Typical CNN Structure

$$\text{Input} \to [\text{CONV} \to \text{ReLU} \to \text{POOL}] \times N \to \text{FC} \to \text{Softmax}$$

8.2 Example: Simple CNN for MNIST

Layer	**Output Shape**	**Parameters**
Input	$28 \times 28 \times 1$	0
Conv1 (32 filters, 3×3)	$28 \times 28 \times 32$	$(3 \times 3 \times 1 + 1) \times 32 = 320$
ReLU	$28 \times 28 \times 32$	0
MaxPool (2×2, S=2)	$14 \times 14 \times 32$	0
Conv2 (64 filters, 3×3)	$14 \times 14 \times 64$	$(3 \times 3 \times 32 + 1) \times 64 = 18,496$
ReLU	$14 \times 14 \times 64$	0
MaxPool (2×2, S=2)	$7 \times 7 \times 64$	0
Flatten	3136	0
Dense	128	$3136 \times 128 + 128 = 401,536$
Output (Softmax)	10	$128 \times 10 + 10 = 1,290$
Total		**421,642**

8.3 Key Patterns

- **Spatial size decreases**: $28 \to 14 \to 7$
- **Depth increases**: $1 \to 32 \to 64$
- **Most parameters in FC layers**: $401,536$ out of $421,642$
- **Modern networks**: Use Global Average Pooling instead of FC layers

9 Summary

9.1 Key Formulas

Output Size:

$$W_{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

Parameters in Conv Layer:

$$\text{Params} = (K \times K \times C_{in} + 1) \times C_{out}$$

(The +1 is for bias)

Backprop - Kernel Gradient:

$$\frac{\partial L}{\partial K_{m,n,d,c}} = \sum_{i,j} X_{i+m,j+n,d} \cdot \frac{\partial L}{\partial Y_{i,j,c}}$$

Backprop - Pooling Gradient:

$$\frac{\partial L}{\partial X} = \begin{cases} \frac{\partial L}{\partial Y} & \text{at max position} \\ 0 & \text{elsewhere} \end{cases}$$

9.2 Design Principles

1. Use small kernels (3×3 or 5×5)
2. Use SAME padding to preserve dimensions
3. Stack multiple CONV layers before pooling
4. Gradually increase depth, decrease spatial size
5. Use stride or pooling for downsampling
6. Apply ReLU after every CONV layer

10 Detailed Padding Examples

10.1 Padding Calculation for Common Cases

Goal: Maintain input size (SAME padding with stride 1)

For output size to equal input size: $W_{out} = W$

$$W = \left\lfloor \frac{W - K + 2P}{1} \right\rfloor + 1$$

$$W - 1 = W - K + 2P \implies 2P = K - 1 \implies P = \frac{K-1}{2}$$

Examples:

- $K = 3$: $P = (3-1)/2 = 1$
- $K = 5$: $P = (5-1)/2 = 2$
- $K = 7$: $P = (7-1)/2 = 3$

This is why odd kernel sizes are preferred!

10.2 Padding with Stride ¿ 1

Example 1: $W = 7, K = 3, P = 1, S = 2$

$$W_{out} = \left\lfloor \frac{7 - 3 + 2(1)}{2} \right\rfloor + 1 = \left\lfloor \frac{6}{2} \right\rfloor + 1 = 3 + 1 = 4$$

Example 2: $W = 224, K = 7, P = 3, S = 2$ (AlexNet first layer)

$$W_{out} = \left\lfloor \frac{224 - 7 + 2(3)}{2} \right\rfloor + 1 = \left\lfloor \frac{223}{2} \right\rfloor + 1 = 111 + 1 = 112$$

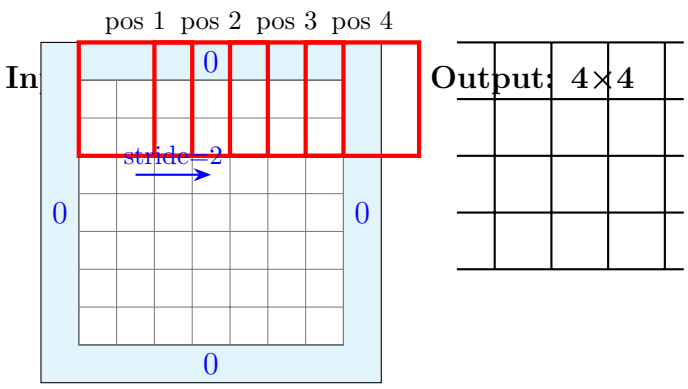


Figure 7: Padding=1, Stride=2: Input 7×7 produces Output 4×4

11 Advanced Convolution Visualization

11.1 Receptive Field Growth

The **receptive field** is the region of the input that affects a particular neuron.

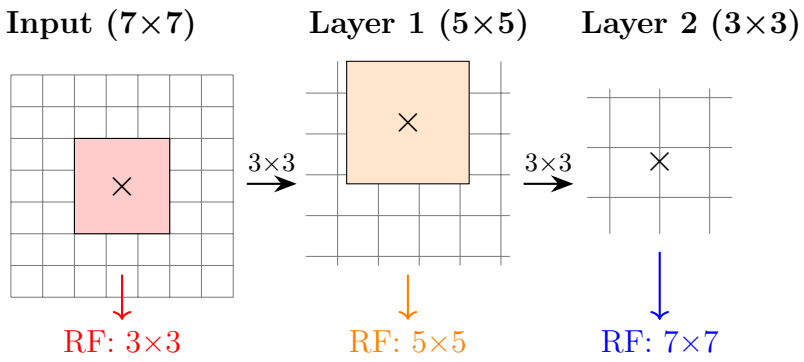


Figure 8: Each layer sees larger region: Receptive field grows with depth

Receptive Field Formula:

$$RF_l = RF_{l-1} + (K_l - 1) \times \prod_{i=1}^{l-1} S_i$$

For stride-1 convolutions:

$$RF_l = 1 + \sum_{i=1}^{l} (K_i - 1)$$

Example: Three 3×3 layers

$$RF = 1 + (3-1) + (3-1) + (3-1) = 1 + 2 + 2 + 2 = 7$$

Three 3×3 convolutions have same receptive field as one 7×7, but with fewer parameters!

11.2 Parameter Comparison

One 7×7 convolution:

$$\text{Params} = 7 \times 7 \times C = 49C$$

Three 3×3 convolutions:

$$\text{Params} = 3 \times (3 \times 3 \times C) = 27C$$

$\implies$ **45% fewer parameters** with same receptive field!

12 Complete Backpropagation Example

12.1 Simple 2-Layer Network

Architecture:

$$\text{Input } X \xrightarrow{\text{Conv}} Z_1 \xrightarrow{\text{ReLU}} A_1 \xrightarrow{\text{MaxPool}} P_1 \xrightarrow{\text{Loss}} L$$

12.2 Forward Pass Equations

Layer 1: Convolution

$$Z_1[i,j,c] = \sum_{m,n,d} X[i+m, j+n, d] \cdot W_1[m,n,d,c] + b_1[c]$$

Layer 2: ReLU

$$A_1[i,j,c] = \max(0, Z_1[i,j,c])$$

Layer 3: Max Pooling

$$P_1[i,j,c] = \max_{m,n \in \text{window}} A_1[i \cdot S + m, j \cdot S + n, c]$$

Loss (e.g., MSE for simplicity)

$$L = \frac{1}{2} \sum_{i,j,c} (P_1[i,j,c] - Y_{\text{true}}[i,j,c])^2$$

12.3 Backward Pass Equations

Step 1: Gradient w.r.t Pooling Output

$$\frac{\partial L}{\partial P_1[i,j,c]} = P_1[i,j,c] - Y_{\text{true}}[i,j,c]$$

Step 2: Gradient through Max Pool

$$\frac{\partial L}{\partial A_1[i,j,c]} = \begin{cases} \frac{\partial L}{\partial P_1[i',j',c]} & \text{if } A_1[i,j,c] \text{ was max in its pool window} \\ 0 & \text{otherwise} \end{cases}$$

Step 3: Gradient through ReLU

$$\frac{\partial L}{\partial Z_1[i,j,c]} = \begin{cases} \frac{\partial L}{\partial A_1[i,j,c]} & \text{if } Z_1[i,j,c] > 0 \\ 0 & \text{if } Z_1[i,j,c] \leq 0 \end{cases}$$

Step 4: Gradient w.r.t Weights

$$\frac{\partial L}{\partial W_1[m,n,d,c]} = \sum_{i,j} X[i+m, j+n, d] \cdot \frac{\partial L}{\partial Z_1[i,j,c]}$$

Step 5: Gradient w.r.t Bias

$$\frac{\partial L}{\partial b_1[c]} = \sum_{i,j} \frac{\partial L}{\partial Z_1[i,j,c]}$$

Step 6: Weight Update (SGD)

$$W_1 \leftarrow W_1 - \eta \cdot \frac{\partial L}{\partial W_1}$$

$$b_1 \leftarrow b_1 - \eta \cdot \frac{\partial L}{\partial b_1}$$

where η is the learning rate.

13 Numerical Example: Small CNN

13.1 Setup

- Input: $4 \times 4 \times 1$ (grayscale)
- Kernel: $2 \times 2 \times 1$, no padding, stride 1
- Expected output: $3 \times 3 \times 1$

13.2 Forward Pass

Input X:

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

Kernel W:

$$W = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Computation at position (0,0):

$$Z[0,0] = (1 \cdot 1) + (2 \cdot 0) + (5 \cdot 0) + (6 \cdot -1) = 1 - 6 = -5$$

Computation at position (0,1):

$$Z[0,1] = (2 \cdot 1) + (3 \cdot 0) + (6 \cdot 0) + (7 \cdot -1) = 2 - 7 = -5$$

Computation at position (0,2):

$$Z[0,2] = (3 \cdot 1) + (4 \cdot 0) + (7 \cdot 0) + (8 \cdot -1) = 3 - 8 = -5$$

Complete output Z:

$$Z = \begin{bmatrix} -5 & -5 & -5 \\ -5 & -5 & -5 \\ -5 & -5 & -5 \end{bmatrix}$$

13.3 After ReLU

$$A = \text{ReLU}(Z) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

All negative values become zero!

13.4 Backward Pass

Assume loss gradient: $\frac{\partial L}{\partial A} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

Through ReLU: Since all $Z[i, j] < 0$:

$$\frac{\partial L}{\partial Z} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Gradient is killed by ReLU! This is the "dying ReLU" problem.

Gradient w.r.t weights:

$$\frac{\partial L}{\partial W} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

No learning happens because ReLU blocked all gradients.

14 Practical Implementation Tips

14.1 Common Hyperparameters

Hyperparameter	Typical Values	Notes
Kernel size	3×3, 5×5	3×3 most common
Padding	SAME or VALID	SAME preserves size
Stride	1, 2	1 for conv, 2 for downsample
Pooling window	2×2, 3×3	2×2 most common
Pooling stride	2	Usually same as window
Learning rate	0.001, 0.0001	Use Adam optimizer
Batch size	32, 64, 128	Depends on GPU memory

14.2 Architecture Design Rules

Rule 1: Gradual Downsampling

$$224 \times 224 \to 112 \times 112 \to 56 \times 56 \to 28 \times 28 \to 14 \times 14 \to 7 \times 7$$

Rule 2: Increasing Depth

$$3 \to 64 \to 128 \to 256 \to 512 \to 1024$$

Rule 3: Channel Count Powers of 2

Use 32, 64, 128, 256, 512 for GPU efficiency

Rule 4: Receptive Field Coverage

Ensure final receptive field covers enough of input for task

14.3 Debugging Checklist

- **Check output shapes**: Use formula $\lfloor \frac{W-K+2P}{S} \rfloor + 1$
- **Verify parameter count**: Should match expected values

- **Monitor activation distributions**: Use tensorboard/wandb
- **Check for dying ReLU**: If many neurons output 0, consider LeakyReLU
- **Validate gradients**: Ensure gradients flow to all layers
- **Test on single batch**: Overfit on 1 batch first

15 Quick Reference Tables

15.1 Output Size Quick Reference

Input	Kernel	Padding	Stride	Output
32	3	0	1	30
32	3	1	1	32
32	5	2	1	32
32	3	0	2	15
64	7	3	2	32
224	7	3	2	112
28	2	0	2	14

15.2 Receptive Field Quick Reference

Network	Receptive Field
1× (3×3)	3
2× (3×3)	5
3× (3×3)	7
4× (3×3)	9
1× (5×5)	5
2× (5×5)	9
1× (7×7)	7

16 Classic CNN Architectures

16.1 LeNet-5 (1998)

Context: One of the first successful CNNs, developed by Yann LeCun for handwritten digit recognition (MNIST dataset).

16.1.1 LeNet-5 Architecture

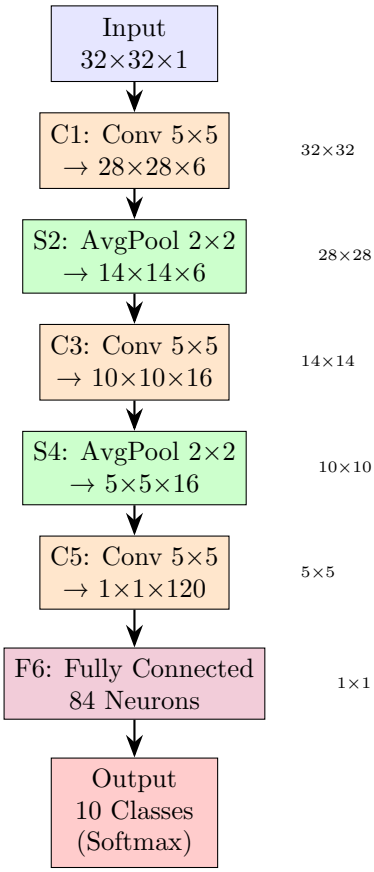


Figure 9: LeNet-5 Architecture — Vertical Flow Representation

16.1.2 Layer-by-Layer Breakdown

Layer	Type	Output	Kernel	Params
Input	-	32×32×1	-	0
C1	Conv	28×28×6	5×5	$(5 \times 5 \times 1 + 1) \times 6 = 156$
S2	AvgPool	14×14×6	2×2, S=2	$6 \times 2 = 12$
C3	Conv	10×10×16	5×5	$(5 \times 5 \times 6 + 1) \times 16 = 2,416$
S4	AvgPool	5×5×16	2×2, S=2	$16 \times 2 = 32$
C5	Conv	1×1×120	5×5	$(5 \times 5 \times 16 + 1) \times 120 = 48,120$
F6	FC	84	-	$120 \times 84 + 84 = 10,164$
Output	FC	10	-	$84 \times 10 + 10 = 850$
Total Parameters				**61,750**

16.1.3 Key Innovations

- **First practical CNN**: Demonstrated CNNs could work on real problems
- **Tanh activation**: Used tanh instead of modern ReLU
- **Average pooling**: Used average pooling with learnable parameters
- **Small network**: Only 60K parameters, trained on CPU

16.1.4 Historical Impact

LeNet-5 was used by banks for check reading in the 1990s, processing millions of checks. It proved CNNs were viable for commercial applications.

16.2 AlexNet (2012)

Context: Won ImageNet 2012 competition with 15.3% error (vs 26.2% second place). Started the deep learning revolution.

16.2.1 AlexNet Architecture

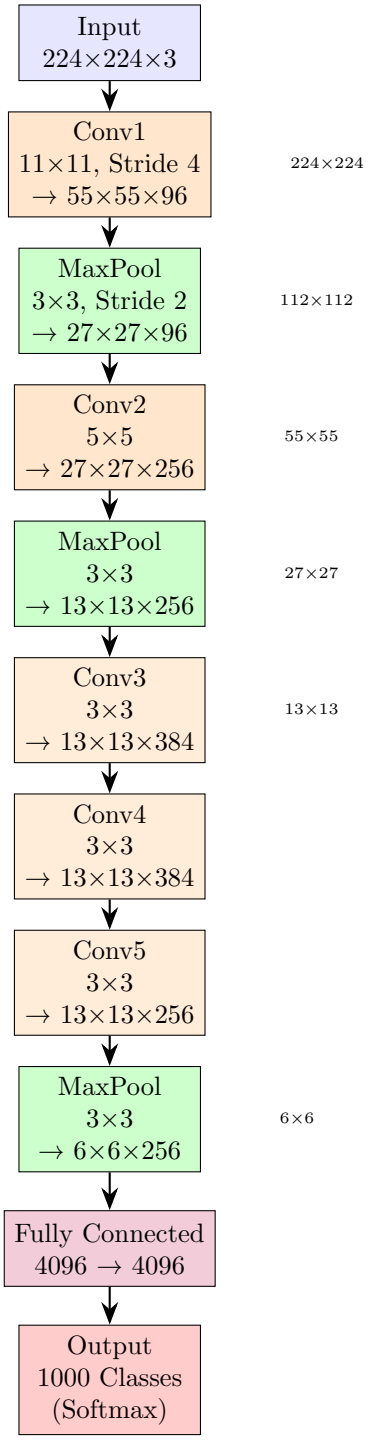


Figure 10: AlexNet Architecture — Vertical Layer Flow

16.2.2 Detailed Layer Specifications

Layer	Output	Kernel	Stride	Pad	Params
Input	224×224×3	-	-	-	0
Conv1	55×55×96	11×11	4	0	35K
MaxPool1	27×27×96	3×3	2	0	0
Conv2	27×27×256	5×5	1	2	614K
MaxPool2	13×13×256	3×3	2	0	0
Conv3	13×13×384	3×3	1	1	885K
Conv4	13×13×384	3×3	1	1	1.33M
Conv5	13×13×256	3×3	1	1	885K
MaxPool5	6×6×256	3×3	2	0	0
FC6	4096	-	-	-	37.7M
FC7	4096	-	-	-	16.8M
FC8	1000	-	-	-	4.1M
Total Parameters					**62.3M**

16.2.3 Key Innovations

- **ReLU activation**: First major use of ReLU instead of tanh/sigmoid
- **Dropout**: 0.5 dropout in FC layers to prevent overfitting
- **Data augmentation**: Random crops, flips, color jittering
- **GPU training**: Trained on 2 GTX 580 GPUs (split architecture)
- **Local Response Normalization (LRN)**: Now obsolete, replaced by Batch Norm
- **Overlapping pooling**: 3×3 window with stride 2

16.3 VGGNet (2014)

Context: Developed by Visual Geometry Group (Oxford). Demonstrated that depth matters — used only 3×3 kernels but went very deep.

16.3.1 VGG-16 Architecture

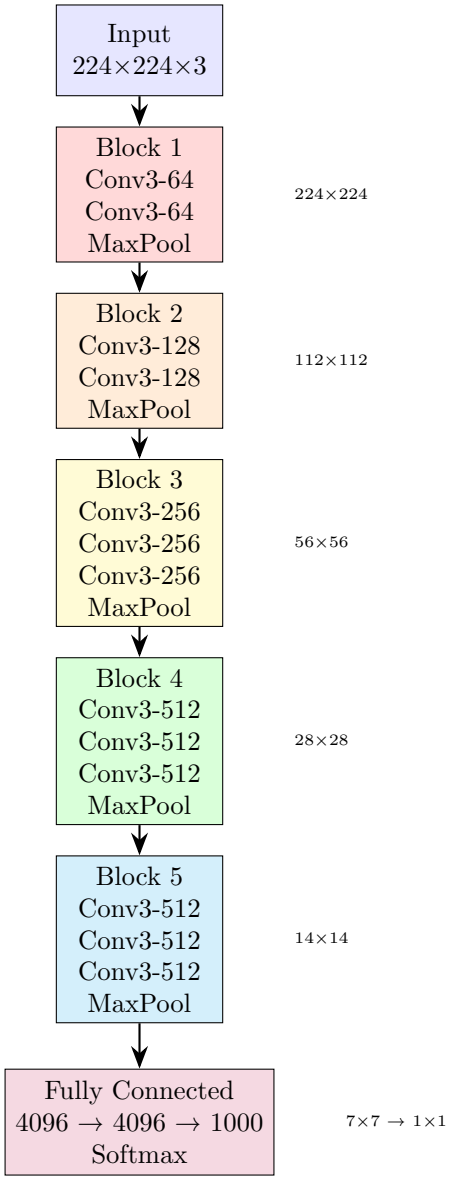


Figure 11: VGG-16 Architecture — Vertical Block Flow

16.3.2 Complete VGG-16 Specifications

Layer	Output Shape	Filters	Parameters
Input	224×224×3	-	0
Conv1-1	224×224×64	3×3, 64	1,792
Conv1-2	224×224×64	3×3, 64	36,928
MaxPool1	112×112×64	2×2, S=2	0
Conv2-1	112×112×128	3×3, 128	73,856
Conv2-2	112×112×128	3×3, 128	147,584
MaxPool2	56×56×128	2×2, S=2	0
Conv3-1	56×56×256	3×3, 256	295,168
Conv3-2	56×56×256	3×3, 256	590,080
Conv3-3	56×56×256	3×3, 256	590,080
MaxPool3	28×28×256	2×2, S=2	0
Conv4-1	28×28×512	3×3, 512	1,180,160
Conv4-2	28×28×512	3×3, 512	2,359,808
Conv4-3	28×28×512	3×3, 512	2,359,808
MaxPool4	14×14×512	2×2, S=2	0
Conv5-1	14×14×512	3×3, 512	2,359,808
Conv5-2	14×14×512	3×3, 512	2,359,808
Conv5-3	14×14×512	3×3, 512	2,359,808
MaxPool5	7×7×512	2×2, S=2	0
Flatten	25,088	-	0
FC1	4096	-	102,764,544
FC2	4096	-	16,781,312
FC3	1000	-	4,097,000
Total Parameters			**138.3M**

16.3.3 VGG Design Philosophy

Core Principle: Use only 3×3 convolutions

Why 3×3 kernels?

- **Smaller receptive field per layer**: But stack them to get large effective receptive field
- **More non-linearity**: Each 3×3 conv followed by ReLU adds non-linearity
- **Fewer parameters**: Two 3×3 convs have fewer params than one 5×5

Mathematical Comparison:

- **One 5×5 conv**: $5 \times 5 \times C^2 = 25C^2$ parameters
- **Two 3×3 convs**: $2 \times (3 \times 3 \times C^2) = 18C^2$ parameters
- **Savings**: $(25 - 18)/25 = 28\%$ fewer parameters

- **Receptive field**: Both have 5×5 effective receptive field

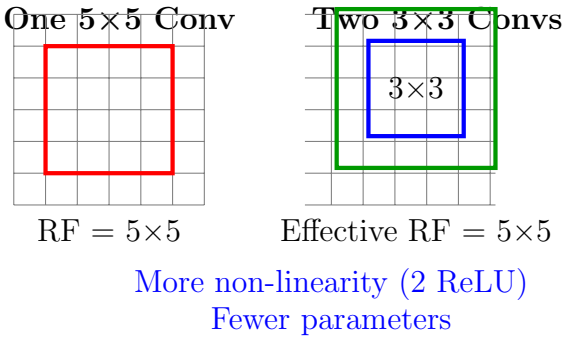


Figure 12: VGG's 3×3 stacking strategy vs single large kernel

16.3.4 VGG Variants

Model	Layers	Parameters	Top-5 Error
VGG-11	11	132.9M	10.4%
VGG-13	13	133.0M	9.9%
VGG-16	16	138.3M	8.8%
VGG-19	19	143.7M	8.8%

16.3.5 Key Characteristics

- **Uniform architecture**: Very regular, easy to implement
- **Deep**: 16-19 weight layers
- **3×3 convolutions everywhere**: With padding=1 to preserve size
- **2×2 max pooling**: After each block to downsample
- **Channel doubling**: 64 → 128 → 256 → 512 → 512
- **Many parameters**: 138M parameters (mostly in FC layers)

16.3.6 Limitations of VGG

- **Memory intensive**: 138M parameters, needs 528MB for weights alone
- **Slow**: Many operations due to large FC layers
- **Harder to train**: No skip connections (addressed by ResNet)

16.3.7 Legacy and Impact

- **Simplicity**: Very simple, uniform architecture
- **Transfer learning**: VGG features work well for other tasks
- **Standard benchmark**: Still used as baseline in many papers
- **Influenced designs**: Led to "deeper is better" trend

17 Detailed Convolution Mathematics

17.1 Cross-Correlation vs Convolution

What we actually use (Cross-Correlation):

$$(I \star K)[i,j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I[i+m, j+n] \cdot K[m,n]$$

True mathematical convolution:

$$(I * K)[i,j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I[i-m, j-n] \cdot K[m,n]$$

The difference is kernel flipping (180° rotation). Since kernels are learned, this distinction doesn't matter in practice.

17.2 Detailed Gradient Derivation

17.2.1 Setup

Consider a simple network:

$$Y = \text{Conv}(X, W) \quad \text{where} \quad Y[i,j] = \sum_{m,n} X[i+m, j+n] \cdot W[m,n]$$

Loss: $L = f(Y)$ for some loss function f.

17.2.2 Chain Rule Application

By chain rule:

$$\frac{\partial L}{\partial W[m,n]} = \sum_{i,j} \frac{\partial L}{\partial Y[i,j]} \cdot \frac{\partial Y[i,j]}{\partial W[m,n]}$$

From the forward equation:

$$\frac{\partial Y[i,j]}{\partial W[m,n]} = X[i+m, j+n]$$

Therefore:

$$\boxed{\frac{\partial L}{\partial W[m,n]} = \sum_{i,j} \frac{\partial L}{\partial Y[i,j]} \cdot X[i+m, j+n]}$$

This is a **convolution** between input X and gradient $\frac{\partial L}{\partial Y}$!

17.2.3 Gradient w.r.t Input

Similarly:

$$\frac{\partial L}{\partial X[i,j]} = \sum_{m,n} \frac{\partial L}{\partial Y} \cdot \frac{\partial Y}{\partial X[i,j]}$$

Since $X[i,j]$ appears in multiple outputs, we sum over all positions where it contributed:

$$\boxed{\frac{\partial L}{\partial X[i,j]} = \sum_{m,n} \frac{\partial L}{\partial Y[i-m, j-n]} \cdot W[m,n]}$$

This is a **full convolution** (flipped kernel).

17.3 Numerical Gradient Checking Example

Finite difference approximation:

$$\frac{\partial L}{\partial w} \approx \frac{L(w+\epsilon) - L(w-\epsilon)}{2\epsilon}$$

Example: Check gradient for $W[0,0]$

1. Compute L with current weights: L_0
2. Add small $\epsilon = 10^{-5}$ to $W[0,0]$: compute L_+
3. Subtract ϵ from $W[0,0]$: compute L_-
4. Numerical gradient: $\frac{L_+ - L_-}{2\epsilon}$
5. Compare with backprop gradient
6. Relative error should be $< 10^{-5}$

$$\text{Relative Error} = \frac{|\nabla_{\text{numerical}} - \nabla_{\text{backprop}}|}{|\nabla_{\text{numerical}}| + |\nabla_{\text{backprop}}|}$$

17.4 Computational Complexity Analysis

17.4.1 Forward Pass Complexity

For a single convolution layer:

- Input: $H_{in} \times W_{in} \times C_{in}$
- Kernel: $K \times K \times C_{in} \times C_{out}$
- Output: $H_{out} \times W_{out} \times C_{out}$

Number of operations:

$$\text{FLOPs} = H_{out} \times W_{out} \times C_{out} \times (K \times K \times C_{in} \times 2)$$

The factor of 2 accounts for multiply-add operations.
Example: $224 \times 224 \times 3$ input, 64 filters of 3×3

$$\text{FLOPs} = 224 \times 224 \times 64 \times (3 \times 3 \times 3 \times 2) = 1.73 \text{ billion operations}$$

17.4.2 Memory Requirements

Weights:

$$\text{Memory}_{\text{weights}} = K \times K \times C_{in} \times C_{out} \times 4 \text{ bytes}$$

Activations (stored for backprop):

$$\text{Memory}_{\text{activations}} = H_{out} \times W_{out} \times C_{out} \times 4 \text{ bytes}$$

Batch processing with batch size B:

$$\text{Total Memory} = \text{Memory}_{\text{weights}} + B \times \text{Memory}_{\text{activations}}$$

18 Advanced Pooling Techniques

18.1 Stochastic Pooling

Instead of always taking the max, we can randomly sample from the pooling window in proportion to the activation magnitudes.

Forward Pass:

1. Compute probabilities within each pooling window:

$$p_i = \frac{a_i}{\sum_j a_j}$$

2. Randomly sample an index i according to probability p_i.
3. Output the corresponding activation a_i.

Benefits:

- Acts as a form of regularization (similar to dropout).
- Encourages robustness by preventing over-reliance on the strongest neuron.
- Maintains diversity of feature representations.

18.2 Mixed Pooling

Definition: Mixed pooling blends the strengths of max and average pooling dynamically:

$$y = \alpha \cdot \max(x) + (1 - \alpha) \cdot \text{avg}(x)$$

where $\alpha \in [0, 1]$ is either fixed or learnable.

Intuition:

- Max pooling focuses on the most dominant features (edge detection).
- Average pooling retains background and contextual information.
- Mixed pooling combines both — giving adaptability across feature maps.

18.3 LP Pooling

A generalized pooling operator that unifies both max and average pooling under a single formula:

$$y = \left(\frac{1}{N} \sum_i |x_i|^p \right)^{1/p}$$

- For $p = 1$: it becomes **average pooling**.
- For $p \to \infty$: it approximates **max pooling**.

This allows flexible control over how aggressively the pooling layer focuses on high activations. Intermediate values ($p = 2, 3$) offer a smooth trade-off between sensitivity and stability.

18.4 Spatial Pyramid Pooling (SPP)

Problem: Fully connected layers require fixed-size inputs, but CNNs can process variable-sized images.

Solution: Spatial Pyramid Pooling (SPP) converts feature maps of arbitrary size into a fixed-length representation.

Method:

1. Divide the feature map into spatial bins at multiple pyramid levels (e.g., 1×1, 2×2, 4×4).
2. Apply max-pooling in each bin.
3. Concatenate all pooled outputs into a single vector.

Result: A fixed-dimensional feature vector regardless of input size.

18.5 Adaptive Pooling

Goal: Obtain a desired output size regardless of input shape.

Instead of specifying kernel size and stride, you define the target output dimensions (H_{out}, W_{out}), and the framework automatically computes suitable stride and padding:

$$\text{stride} = \left\lfloor \frac{H_{in}}{H_{out}} \right\rfloor, \quad \text{kernel size} = H_{in} - (H_{out} - 1) \times \text{stride}$$

Use Case: Commonly used before fully connected layers or global pooling to unify varying input resolutions (e.g., PyTorch's `AdaptiveAvgPool2d((1,1))`).

18.6 Global Context Pooling (GCP)

A modern pooling variant that combines global context aggregation with attention mechanisms.

Concept:

$$y_c = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot \text{GAP}(X_c)))$$

where GAP is global average pooling, W_1, W_2 are learnable weights, and σ is a sigmoid gating function.

Intuition:

- Captures global dependencies across channels.
- Enhances feature reweighting (similar to SE-blocks).
- Improves context awareness in CNNs used for classification and detection.

18.7 Comparison Summary

Table 1: Comparison of Pooling Methods

Pooling Type	**Computation**	**Invariance Type**	**Use Case**
Max Pooling	$\max(x)$	Translation (local)	Feature selection
Average Pooling	$\frac{1}{N}\sum x$	Spatial averaging	Smoothing
Stochastic Pooling	$\text{Random}(x, p)$	Probabilistic	Regularization
Mixed Pooling	$\alpha \max + (1-\alpha)\text{avg}$	Adaptive	Early layers
LP Pooling	$(\frac{1}{N}\sum \lvert x\rvert^p)^{1/p}$	Parametric	Adjustable sensitivity
SPP	Multi-bin pooling	Scale invariance	Object detection
Adaptive Pooling	Dynamic stride	Resolution invariance	Pre-FC compression
GCP	Attention-weighted GAP	Global context	Classification, segmentation

Key Takeaway

- Pooling operations are no longer limited to fixed kernels and rigid structures.
- Modern CNNs use adaptive, stochastic, or attention-weighted pooling to capture multi-scale information and enhance robustness against translation, scaling, and deformation.

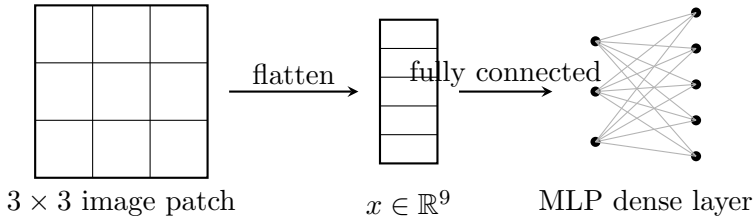
Part II
CNN Backpropagation Notes (With Matrix Layouts)

19 Why a Plain MLP Fails on Raw Image Data

Main issue 1: parameter explosion. If an image of size $H \times W$ is flattened, an MLP must connect every pixel to every hidden neuron. For a grayscale image, input dimension is HW; for RGB it is $3HW$. This rapidly creates very large weight matrices.

Main issue 2: no spatial inductive bias. Flattening destroys 2D locality. Neighboring pixels in the image are no longer treated as local neighbors in the same structured way.

Main issue 3: no translation equivariance by design. A cat shifted by a few pixels becomes a very different input vector, so an MLP generally needs to relearn similar patterns at many positions.

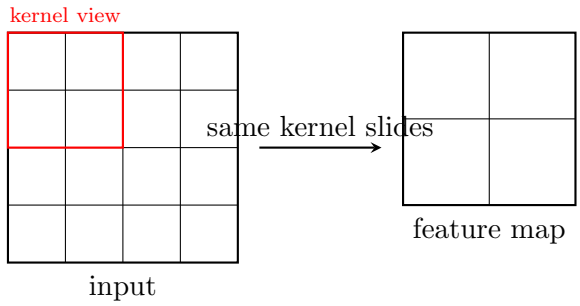


20 How CNN Solves These Issues

Local connectivity: each neuron looks at a small receptive field (e.g., 3×3), preserving locality.

Weight sharing: one kernel is reused across all spatial positions, reducing parameters dramatically.

Translation-friendly representation: convolving the same filter across the image creates feature maps that respond to the same pattern at different locations.



21 From Hubel & Wiesel to Modern CNNs (Short History)

Hubel and Wiesel (1959–1968): visual cortex studies identified cells responding to local oriented edges and increasingly complex patterns across hierarchical stages.

Neocognitron (Fukushima, 1980): introduced hierarchical, shift-tolerant pattern recognition with local receptive fields and shared-like operations.

LeNet-5 (LeCun et al., 1990s): practical gradient-based CNN for digit recognition; established convolution + pooling + fully connected pipeline.

Deep CNN era: AlexNet (2012), VGG, GoogLeNet, ResNet and beyond scaled CNNs with better optimization, regularization, and compute.

Key continuity: local receptive fields + hierarchical feature extraction from simple to complex are direct conceptual descendants of Hubel–Wiesel ideas.

22 Layer-0 Input, Kernel, and Layer-1 Output

Why this setup? In convolution, each output value is produced by overlaying the kernel on a local patch of the input, multiplying element-wise, and summing. Because the input

is 5×5, kernel is 3×3, and stride is 2, the kernel can be placed in exactly four valid positions, which gives a 2×2 output map.

$$\text{Layer-0 (input) } \boldsymbol{A} = \begin{bmatrix} a_1 & a_2 & a_3 & a_4 & a_5 \\ a_6 & a_7 & a_8 & a_9 & a_{10} \\ a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{16} & a_{17} & a_{18} & a_{19} & a_{20} \\ a_{21} & a_{22} & a_{23} & a_{24} & a_{25} \end{bmatrix}$$

$$\text{Kernel (size } 3 \times 3) \ \boldsymbol{W} = \begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix}$$

$$\text{Layer-1 output } \boldsymbol{Z} = \begin{bmatrix} z_1 & z_2 \\ z_3 & z_4 \end{bmatrix}, \qquad \text{stride} = 2.$$

Equivalent vector form used in your notes:

$$\boldsymbol{z} = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{bmatrix}.$$

Why both matrix and vector forms? The matrix form preserves spatial layout, while the vector form is convenient for chain-rule notation and compact derivative expressions.

23 Forward Matrices (the 4 receptive fields)

How these four patch matrices appear: with stride 2, the top-left corner of the 3×3 kernel moves by two columns/rows each step: positions $(1,1)$, $(1,3)$, $(3,1)$, and $(3,3)$ in the input index grid. Those four placements are exactly the four matrices shown below.

$$\underbrace{\begin{bmatrix} a_1 & a_2 & a_3 \\ a_6 & a_7 & a_8 \\ a_{11} & a_{12} & a_{13} \end{bmatrix}}_{\text{for } z_1}, \quad \underbrace{\begin{bmatrix} a_3 & a_4 & a_5 \\ a_8 & a_9 & a_{10} \\ a_{13} & a_{14} & a_{15} \end{bmatrix}}_{\text{for } z_2},$$

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{16} & a_{17} & a_{18} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}}_{\text{for } z_3}, \quad \underbrace{\begin{bmatrix} a_{13} & a_{14} & a_{15} \\ a_{18} & a_{19} & a_{20} \\ a_{23} & a_{24} & a_{25} \end{bmatrix}}_{\text{for } z_4}.$$

$$\begin{aligned}
z_1 &= a_1w_1 + a_2w_2 + a_3w_3 + a_6w_4 + a_7w_5 + a_8w_6 + a_{11}w_7 + a_{12}w_8 + a_{13}w_9, \\
z_2 &= a_3w_1 + a_4w_2 + a_5w_3 + a_8w_4 + a_9w_5 + a_{10}w_6 + a_{13}w_7 + a_{14}w_8 + a_{15}w_9, \\
z_3 &= a_{11}w_1 + a_{12}w_2 + a_{13}w_3 + a_{16}w_4 + a_{17}w_5 + a_{18}w_6 + a_{21}w_7 + a_{22}w_8 + a_{23}w_9, \\
z_4 &= a_{13}w_1 + a_{14}w_2 + a_{15}w_3 + a_{18}w_4 + a_{19}w_5 + a_{20}w_6 + a_{23}w_7 + a_{24}w_8 + a_{25}w_9.
\end{aligned}$$

Why each z_i has 9 terms: each output scalar comes from a 3×3 patch, so there are 9 element-wise products with the 9 kernel weights.

24 Loss Gradient at Layer-1

Why define $\partial L/\partial \boldsymbol{Z}$ first? In backpropagation, gradients flow from the loss backward. For this layer, the immediate upstream information is how sensitive the loss is to each output activation $z_1, \ldots, z_4$.

$$\boldsymbol{G} = \frac{\partial L}{\partial \boldsymbol{Z}} = \begin{bmatrix} \frac{\partial L}{\partial z_1} & \frac{\partial L}{\partial z_2} \\ \frac{\partial L}{\partial z_3} & \frac{\partial L}{\partial z_4} \end{bmatrix}, \qquad \frac{\partial L}{\partial \boldsymbol{z}} = \begin{bmatrix} \frac{\partial L}{\partial z_1} \\ \frac{\partial L}{\partial z_2} \\ \frac{\partial L}{\partial z_3} \\ \frac{\partial L}{\partial z_4} \end{bmatrix}.$$

25 Matrix Form for $\partial L/\partial \boldsymbol{W}$

How chain rule gives this form: each weight w_j affects every output z_1, z_2, z_3, z_4 (because the same kernel is reused at all spatial locations). So for each weight, gradients from all four outputs must be added.

$$\frac{\partial L}{\partial \boldsymbol{W}} = \frac{\partial L}{\partial z_1} \begin{bmatrix} a_1 & a_2 & a_3 \\ a_6 & a_7 & a_8 \\ a_{11} & a_{12} & a_{13} \end{bmatrix} + \frac{\partial L}{\partial z_2} \begin{bmatrix} a_3 & a_4 & a_5 \\ a_8 & a_9 & a_{10} \\ a_{13} & a_{14} & a_{15} \end{bmatrix} + \frac{\partial L}{\partial z_3} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{16} & a_{17} & a_{18} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} + \frac{\partial L}{\partial z_4} \begin{bmatrix} a_{13} & a_{14} & a_{15} \\ a_{18} & a_{19} & a_{20} \\ a_{23} & a_{24} & a_{25} \end{bmatrix}.$$

Interpretation of the sum: each term is "(local input patch) $\times$ (its output gradient)". Summing four such terms accumulates all contributions to the shared kernel.

Element-wise gradient matrix (same layout as kernel):

$$\frac{\partial L}{\partial \boldsymbol{W}} = \begin{bmatrix} \frac{\partial L}{\partial w_1} & \frac{\partial L}{\partial w_2} & \frac{\partial L}{\partial w_3} \\ \frac{\partial L}{\partial w_4} & \frac{\partial L}{\partial w_5} & \frac{\partial L}{\partial w_6} \\ \frac{\partial L}{\partial w_7} & \frac{\partial L}{\partial w_8} & \frac{\partial L}{\partial w_9} \end{bmatrix}.$$

Why same layout as $\boldsymbol{W}$: the derivative with respect to a matrix has the same shape as that matrix, so each entry aligns with one kernel parameter.

26 Weight Update Matrix

Why this update equation: gradient descent moves parameters in the negative-gradient direction, which decreases loss for a sufficiently small learning rate η.

$$\boldsymbol{W}^* = \boldsymbol{W} - \eta \frac{\partial L}{\partial \boldsymbol{W}} = \begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix} - \eta \begin{bmatrix} \frac{\partial L}{\partial w_1} & \frac{\partial L}{\partial w_2} & \frac{\partial L}{\partial w_3} \\ \frac{\partial L}{\partial w_4} & \frac{\partial L}{\partial w_5} & \frac{\partial L}{\partial w_6} \\ \frac{\partial L}{\partial w_7} & \frac{\partial L}{\partial w_8} & \frac{\partial L}{\partial w_9} \end{bmatrix}.$$

Practical reading: "new kernel = old kernel minus step-size times kernel gradient."

27 Expanded Scalar Gradients (from chain rule)

How to read one line (example w_1):

$$\frac{\partial L}{\partial w_1} = \sum_{k=1}^{4} \frac{\partial L}{\partial z_k} \frac{\partial z_k}{\partial w_1}$$

and from the forward equations,

$$\frac{\partial z_1}{\partial w_1} = a_1, \quad \frac{\partial z_2}{\partial w_1} = a_3, \quad \frac{\partial z_3}{\partial w_1} = a_{11}, \quad \frac{\partial z_4}{\partial w_1} = a_{13},$$

which directly gives the first expanded equation below. The same pattern is applied to $w_2, \ldots, w_9$.

$$\begin{aligned}
\frac{\partial L}{\partial w_1} &= a_1 \frac{\partial L}{\partial z_1} + a_3 \frac{\partial L}{\partial z_2} + a_{11} \frac{\partial L}{\partial z_3} + a_{13} \frac{\partial L}{\partial z_4}, \\
\frac{\partial L}{\partial w_2} &= a_2 \frac{\partial L}{\partial z_1} + a_4 \frac{\partial L}{\partial z_2} + a_{12} \frac{\partial L}{\partial z_3} + a_{14} \frac{\partial L}{\partial z_4}, \\
\frac{\partial L}{\partial w_3} &= a_3 \frac{\partial L}{\partial z_1} + a_5 \frac{\partial L}{\partial z_2} + a_{13} \frac{\partial L}{\partial z_3} + a_{15} \frac{\partial L}{\partial z_4}, \\
\frac{\partial L}{\partial w_4} &= a_6 \frac{\partial L}{\partial z_1} + a_8 \frac{\partial L}{\partial z_2} + a_{16} \frac{\partial L}{\partial z_3} + a_{18} \frac{\partial L}{\partial z_4}, \\
\frac{\partial L}{\partial w_5} &= a_7 \frac{\partial L}{\partial z_1} + a_9 \frac{\partial L}{\partial z_2} + a_{17} \frac{\partial L}{\partial z_3} + a_{19} \frac{\partial L}{\partial z_4}, \\
\frac{\partial L}{\partial w_6} &= a_8 \frac{\partial L}{\partial z_1} + a_{10} \frac{\partial L}{\partial z_2} + a_{18} \frac{\partial L}{\partial z_3} + a_{20} \frac{\partial L}{\partial z_4}, \\
\frac{\partial L}{\partial w_7} &= a_{11} \frac{\partial L}{\partial z_1} + a_{13} \frac{\partial L}{\partial z_2} + a_{21} \frac{\partial L}{\partial z_3} + a_{23} \frac{\partial L}{\partial z_4}, \\
\frac{\partial L}{\partial w_8} &= a_{12} \frac{\partial L}{\partial z_1} + a_{14} \frac{\partial L}{\partial z_2} + a_{22} \frac{\partial L}{\partial z_3} + a_{24} \frac{\partial L}{\partial z_4}, \\
\frac{\partial L}{\partial w_9} &= a_{13} \frac{\partial L}{\partial z_1} + a_{15} \frac{\partial L}{\partial z_2} + a_{23} \frac{\partial L}{\partial z_3} + a_{25} \frac{\partial L}{\partial z_4}.
\end{aligned}$$

28 Padding, Stride, and Output Size Formula

For 1D intuition (same idea per axis in 2D):

$$\text{output length} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1,$$

where

- n: input size,
- k: kernel size,
- p: zero-padding on each side,
- s: stride.

Special case: $p = 0$, $s = 1$ gives

$$\text{out} = n - k + 1.$$

For 2D, apply per dimension:

$$\text{out}_h = \left\lfloor \frac{H + 2p_h - k_h}{s_h} \right\rfloor + 1, \qquad \text{out}_w = \left\lfloor \frac{W + 2p_w - k_w}{s_w} \right\rfloor + 1.$$

Example (your setup): $H = W = 5$, $k_h = k_w = 3$, $p_h = p_w = 0$, $s_h = s_w = 2$:

$$\text{out}_h = \text{out}_w = \left\lfloor \frac{5-3}{2} \right\rfloor + 1 = 2.$$

Hence output is 2×2.

29 Backpropagation Through This Convolution Layer

Given: upstream gradient map

$$\frac{\partial L}{\partial \boldsymbol{Z}} = \begin{bmatrix} \frac{\partial L}{\partial z_1} & \frac{\partial L}{\partial z_2} \\ \frac{\partial L}{\partial z_3} & \frac{\partial L}{\partial z_4} \end{bmatrix}.$$

Step A (already derived): compute $\partial L / \partial \boldsymbol{W}$ by summing input patches weighted by corresponding $\partial L / \partial z_i$.

Step B (optional, gradient wrt input): each input element accumulates contributions from all outputs whose receptive fields include that input. In compact form, this is convolution of the padded upstream gradient with the spatially flipped kernel.

Step C (parameter update):

$$\boldsymbol{W}^{(t+1)} = \boldsymbol{W}^{(t)} - \eta \frac{\partial L}{\partial \boldsymbol{W}}.$$

Step D (repeat over batches/epochs): this iterative update learns filters that minimize training loss.

30 Complete CNN Architecture (Part 2 Reference)

$$\text{Input} \to [\text{CONV} \to \text{ReLU} \to \text{POOL}] \times N \to \text{FC} \to \text{Softmax}$$

Example: Simple CNN for MNIST

Layer	**Output Shape**	**Parameters**
Input	$28 \times 28 \times 1$	0
Conv1 (32 filters, 3×3)	$28 \times 28 \times 32$	$(3 \times 3 \times 1 + 1) \times 32 = 320$
MaxPool (2×2, S=2)	$14 \times 14 \times 32$	0
Conv2 (64 filters, 3×3)	$14 \times 14 \times 64$	$(3 \times 3 \times 32 + 1) \times 64 = 18{,}496$
MaxPool (2×2, S=2)	$7 \times 7 \times 64$	0
Flatten	3136	0
Dense	128	$3136 \times 128 + 128 = 401{,}536$
Output (Softmax)	10	$128 \times 10 + 10 = 1{,}290$
Total		**421,642**

31 Backpropagation in CNNs (General Multi-Channel Form)

Given input $X \in \mathbb{R}^{H \times W \times C_{in}}$, kernel $K \in \mathbb{R}^{k_H \times k_W \times C_{in} \times C_{out}}$, output $Y \in \mathbb{R}^{H' \times W' \times C_{out}}$:

$$Y_{i,j,c} = \sum_{m=0}^{k_H-1} \sum_{n=0}^{k_W-1} \sum_{d=0}^{C_{in}-1} X_{i+m,j+n,d} \, K_{m,n,d,c} + b_c.$$

Kernel gradient:

$$\frac{\partial L}{\partial K_{m,n,d,c}} = \sum_{i=0}^{H'-1} \sum_{j=0}^{W'-1} X_{i+m,j+n,d} \frac{\partial L}{\partial Y_{i,j,c}}.$$

Input gradient (conceptually full conv with flipped kernel):

$$\frac{\partial L}{\partial X_{i,j,d}} = \sum_{m=0}^{k_H-1} \sum_{n=0}^{k_W-1} \sum_{c=0}^{C_{out}-1} K_{m,n,d,c} \frac{\partial L}{\partial Y_{i-m,j-n,c}}.$$

Bias gradient:

$$\frac{\partial L}{\partial b_c} = \sum_{i=0}^{H'-1} \sum_{j=0}^{W'-1} \frac{\partial L}{\partial Y_{i,j,c}}.$$

32 Pooling Backprop and ReLU Backprop

Max-pooling backprop: gradient goes only to the max location recorded in forward pass.

$$\frac{\partial L}{\partial X_{i,j,d}} = \begin{cases} \frac{\partial L}{\partial Y_{i',j',d}}, & \text{if } X_{i,j,d} \text{ was max in its window} \\ 0, & \text{otherwise} \end{cases}$$

ReLU:

$$y = \max(0, x), \qquad \frac{\partial L}{\partial x} = \begin{cases} \frac{\partial L}{\partial y}, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

33 Detailed Padding Examples (Part 2)

For SAME-style size preservation at stride 1:

$$W_{out} = W \Rightarrow W = \left\lfloor \frac{W - K + 2P}{1} \right\rfloor + 1 \Rightarrow P = \frac{K-1}{2}.$$

Examples: $K = 3 \Rightarrow P = 1$, $K = 5 \Rightarrow P = 2$, $K = 7 \Rightarrow P = 3$.

With stride > 1:

$$W = 7, K = 3, P = 1, S = 2 \Rightarrow W_{out} = \left\lfloor \frac{7 - 3 + 2}{2} \right\rfloor + 1 = 4.$$

$$W = 224, K = 7, P = 3, S = 2 \Rightarrow W_{out} = \left\lfloor \frac{224 - 7 + 6}{2} \right\rfloor + 1 = 112.$$

34 Receptive Field Growth

For layer l:

$$RF_l = RF_{l-1} + (K_l - 1)\prod_{i=1}^{l-1} S_i.$$

For all strides = 1:

$$RF_l = 1 + \sum_{i=1}^{l}(K_i - 1).$$

Example: three 3×3 layers give

$$RF = 1 + 2 + 2 + 2 = 7.$$

Hence stacked small kernels can match large receptive fields with fewer parameters.

35 Classic CNN Architectures Summary (Part 2)

LeNet-5 (1998): early practical CNN for digit recognition.

Layer	Output	Params
C1 (5×5, 6)	28×28×6	156
S2 (AvgPool)	14×14×6	12
C3 (5×5, 16)	10×10×16	2,416
S4 (AvgPool)	5×5×16	32
C5 (5×5,120)	1×1×120	48,120
F6 + Output	84 & 10	11,014
Total		61,750

AlexNet (2012): deep CNN breakthrough on ImageNet, ReLU + dropout + GPU training.

VGG-16 (2014): uniform 3×3 convolutions, deeper stacks. Two 3×3 layers vs one 5×5:

$$2 \times (3 \times 3 \times C^2) = 18C^2 \ < \ 25C^2 = (5 \times 5 \times C^2).$$

So stacked 3×3 can be more parameter-efficient.

36 Numerical Mini Example and Dying ReLU

Input

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Then each 2×2 patch gives −5, so

$$Z = \begin{bmatrix} -5 & -5 & -5 \\ -5 & -5 & -5 \\ -5 & -5 & -5 \end{bmatrix}, \quad A = \mathrm{ReLU}(Z) = 0.$$

If upstream gradient at A is all ones, gradient through ReLU is all zeros (because all pre-activations are negative), so weight gradients become zero: this illustrates the dying-ReLU issue.

37 Advanced Pooling Variants

Stochastic pooling: sample within window using probabilities

$$p_i = \frac{a_i}{\sum_j a_j}.$$

Mixed pooling:

$$y = \alpha \max(x) + (1-\alpha)\operatorname{avg}(x), \quad \alpha \in [0,1].$$

L_p **pooling:**

$$y = \left(\frac{1}{N}\sum_i |x_i|^p\right)^{1/p},$$

with $p = 1$ giving average pooling and large p approaching max pooling.

SPP: multi-level bins (e.g., 1×1, 2×2, 4×4) then concatenate pooled outputs to get fixed-length representations from variable-size inputs.

Adaptive pooling: choose output size first, framework computes effective stride/window.

Global-context/attention pooling: channel-wise global summary + learned gating for reweighting.

38 Practical Tips and Quick Reference

Common conv output formula:

$$W_{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1.$$

Conv parameter count:

$$\text{Params} = (K \times K \times C_{in} + 1) \times C_{out}.$$

Debug checklist:

- verify output shapes layer-by-layer,
- verify parameter counts,
- monitor activation distributions,
- check for dying ReLU,
- test single-batch overfitting,
- confirm non-zero gradients in early layers.